

## **BANKING SCENARIO**

### **Basic + WHERE + Operators**

1. Retrieve all accounts with balance greater than 20,000:

```
SELECT * FROM accounts WHERE balance > 20000;
```

2. Find customers who live in Chennai:

```
SELECT * FROM customers WHERE city = 'Chennai';
```

3. Display accounts with balance between 20,000 and 50,000:

```
SELECT * FROM accounts WHERE balance BETWEEN 20000 AND 50000;
```

4. Find customers whose names start with 'J':

```
SELECT * FROM customers WHERE name LIKE 'J%';
```

5. Retrieve accounts of type 'Savings' or 'Current':

```
SELECT * FROM accounts WHERE account_type IN ('Savings', 'Current');
```

6. Display accounts that are not 'Savings':

```
SELECT * FROM accounts WHERE account_type != 'Savings';
```

7. Find customers whose names contain the letter 'a':

```
SELECT * FROM customers WHERE name LIKE '%a%';
```

8. Retrieve accounts with balance less than or equal to 30,000:

```
SELECT * FROM accounts WHERE balance <= 30000;
```

9. Find customers who are not from Madurai:

```
SELECT * FROM customers WHERE city != 'Madurai';
```

10. Display accounts where balance is not between 10,000 and 40,000:

```
SELECT * FROM accounts WHERE balance NOT BETWEEN 10000 AND 40000;
```

11. Retrieve customers whose names end with 'i':

```
SELECT * FROM customers WHERE name LIKE '%i';
```

12. Find accounts with balance equal to 50,000:

```
SELECT * FROM accounts WHERE balance = 50000;
```

13. Display customers whose city is either Chennai or Salem:

```
SELECT * FROM customers WHERE city IN ('Chennai', 'Salem');
```

14. Find accounts with balance greater than 10,000 and less than 40,000:

```
SELECT * FROM accounts WHERE balance > 10000 AND balance < 40000;
```

15. Retrieve accounts where account type is not in ('Current'):

```
SELECT * FROM accounts WHERE account_type NOT IN ('Current');
```

## **ORDER BY**

16. Display all accounts sorted by balance in descending order:

```
SELECT * FROM accounts ORDER BY balance DESC;
```

17. List customers sorted alphabetically by name:

```
SELECT * FROM customers ORDER BY name ASC;
```

18. Display accounts sorted by account type and then by balance (descending):

```
SELECT * FROM accounts ORDER BY account_type ASC, balance DESC;
```

## AGGREGATE FUNCTIONS

19. Find the total balance of all accounts:

```
SELECT SUM(balance) AS total_balance FROM accounts;
```

20. Calculate the average balance of accounts:

```
SELECT AVG(balance) AS average_balance FROM accounts;
```

21. Find the maximum account balance:

```
SELECT MAX(balance) AS max_balance FROM accounts;
```

22. Find the minimum account balance:

```
SELECT MIN(balance) AS min_balance FROM accounts;
```

23. Count the total number of customers:

```
SELECT COUNT(*) AS total_customers FROM customers;
```

## GROUP BY + HAVING

24. Find total balance grouped by account type:

```
SELECT account_type, SUM(balance) AS total_balance FROM accounts GROUP BY  
account_type;
```

25. Find average balance for each account type:

```
SELECT account_type, AVG(balance) AS avg_balance FROM accounts GROUP BY  
account_type;
```

26. Display account types having average balance greater than 20,000:

```
SELECT account_type, AVG(balance) AS avg_balance FROM accounts GROUP BY  
account_type HAVING AVG(balance) > 20000;
```

27. Count number of accounts for each customer:

```
SELECT customer_id, COUNT(*) AS account_count FROM accounts GROUP BY customer_id;
```

28. Display customers having more than one account:

```
SELECT customer_id, COUNT(*) AS account_count FROM accounts GROUP BY customer_id  
HAVING COUNT(*) > 1;
```

## JOINS

29. Retrieve customer names along with their account balances:

```
SELECT c.name, a.balance FROM customers c JOIN accounts a ON c.customer_id =  
a.customer_id;
```

30. Display all customers and their accounts (including customers without accounts):

```
SELECT c.name, a.account_id, a.account_type, a.balance FROM customers c LEFT JOIN  
accounts a ON c.customer_id = a.customer_id;
```

31. Display all accounts and corresponding customer details:

```
SELECT a.*, c.name, c.city FROM accounts a JOIN customers c ON a.customer_id =  
c.customer_id;
```

32. Retrieve customer names and account types where balance is greater than 20,000:

```
SELECT c.name, a.account_type FROM customers c JOIN accounts a ON c.customer_id =  
a.customer_id WHERE a.balance > 20000;
```

33. List customers with their total balance using JOIN:

```
SELECT c.name, SUM(a.balance) AS total_balance FROM customers c JOIN accounts a ON  
c.customer_id = a.customer_id GROUP BY c.customer_id, c.name;
```

34. Display customer names and balances sorted by balance:

```
SELECT c.name, a.balance FROM customers c JOIN accounts a ON c.customer_id =  
a.customer_id ORDER BY a.balance ASC;
```

35. Count number of accounts for each city using JOIN:

```
SELECT c.city, COUNT(a.account_id) AS total_accounts FROM customers c JOIN accounts a  
ON c.customer_id = a.customer_id GROUP BY c.city;
```

## **SUBQUERIES**

36. Find accounts with balance greater than average balance:

```
SELECT * FROM accounts WHERE balance > (SELECT AVG(balance) FROM accounts);
```

37. Retrieve customers who have accounts:

```
SELECT * FROM customers WHERE customer_id IN (SELECT DISTINCT customer_id FROM  
accounts);
```

38. Find customers who do not have any accounts:

```
SELECT * FROM customers WHERE customer_id NOT IN (SELECT DISTINCT customer_id  
FROM accounts);
```

39. Display account(s) with the maximum balance:

```
SELECT * FROM accounts WHERE balance = (SELECT MAX(balance) FROM accounts);
```

40. Find customers whose total balance is greater than 40,000:

```
SELECT * FROM customers WHERE customer_id IN (SELECT customer_id FROM accounts  
GROUP BY customer_id HAVING SUM(balance) > 40000);
```

## **SECTION B: RAILWAY RESERVATION SCENARIO**

Basic + WHERE + Operators (41–45)

41. Retrieve all bookings with fare greater than 400:

```
SELECT * FROM bookings WHERE fare > 400;
```

42. Find bookings where status is not 'Confirmed':

```
SELECT * FROM bookings WHERE status != 'Confirmed';
```

43. Display trains starting from Chennai:

```
SELECT * FROM trains WHERE source = 'Chennai';
```

44. Retrieve bookings with fare between 300 and 500:

```
SELECT * FROM bookings WHERE fare BETWEEN 300 AND 500;
```

45. Find passengers whose names start with 'A':

```
SELECT passenger_name FROM bookings WHERE passenger_name LIKE 'A%';
```

## **JOINS + GROUP BY + ORDER BY**

46. Retrieve train names along with passenger names:

```
SELECT t.train_name, b.passenger_name FROM trains t JOIN bookings b ON t.train_id = b.train_id;
```

47. Count number of bookings for each train:

```
SELECT t.train_name, COUNT(b.booking_id) AS total_bookings FROM trains t LEFT JOIN bookings b ON t.train_id = b.train_id GROUP BY t.train_id, t.train_name;
```

48. Display train names and total fare collected for each train:

```
SELECT t.train_name, SUM(b.fare) AS total_fare FROM trains t JOIN bookings b ON t.train_id = b.train_id GROUP BY t.train_id, t.train_name;
```

## SUBQUERIES

49. Find bookings with fare equal to the highest fare:

```
SELECT * FROM bookings WHERE fare = (SELECT MAX(fare) FROM bookings);
```

50. Retrieve trains that have more than one booking:

```
SELECT * FROM trains WHERE train_id IN (SELECT train_id FROM bookings GROUP BY train_id HAVING COUNT(*) > 1);
```

## SECTION C: EMPLOYEE MANAGEMENT

51. Find the total number of employees in each department:

```
SELECT department, COUNT(*) AS total_employees FROM Employee GROUP BY department;
```

52. Find the average salary of employees in each department:

```
SELECT department, AVG(salary) AS avg_salary FROM Employee GROUP BY department;
```

53. Display departments having more than one employee:

```
SELECT department, COUNT(*) AS emp_count FROM Employee GROUP BY department HAVING COUNT(*) > 1;
```

54. Find the highest salary in each department:

```
SELECT department, MAX(salary) AS max_salary FROM Employee GROUP BY department;
```

55. Find the lowest salary in each department:

```
SELECT department, MIN(salary) AS min_salary FROM Employee GROUP BY department;
```

56. Find departments whose average salary is greater than 50,000:

```
SELECT department, AVG(salary) AS avg_salary FROM Employee GROUP BY department HAVING AVG(salary) > 50000;
```

57. Calculate the total salary expenditure for each department:

```
SELECT department, SUM(salary) AS total_expenditure FROM Employee GROUP BY department;
```

58. Display all employees sorted by salary in descending order:

```
SELECT * FROM Employee ORDER BY salary DESC;
```

59. Display employees sorted first by department and then by salary in descending order:

```
SELECT * FROM Employee ORDER BY department ASC, salary DESC;
```

60. Find cities that have more than one employee:

```
SELECT city, COUNT(*) AS emp_count FROM Employee GROUP BY city HAVING COUNT(*) > 1;
```

61. Find the total salary paid in each city:

```
SELECT city, SUM(salary) AS total_salary FROM Employee GROUP BY city;
```

62. Display departments ordered by total salary expenditure from highest to lowest:

```
SELECT department, SUM(salary) AS total_expenditure FROM Employee GROUP BY department ORDER BY total_expenditure DESC;
```

63. Find the number of employees in each department whose salary is greater than 50,000:

```
SELECT department, COUNT(*) AS emp_count FROM Employee WHERE salary > 50000 GROUP BY department;
```

64. Find the difference between the highest and lowest salary in each department:

```
SELECT department, (MAX(salary) - MIN(salary)) AS salary_difference FROM Employee GROUP BY department;
```

65. Display the top 3 highest-paid employees:

```
SELECT * FROM Employee ORDER BY salary DESC LIMIT 3;
```

## **SECTION D: CUSTOMERS & ORDERS TABLES**

66. Find the total order amount for each customer:

```
SELECT customer_id, SUM(amount) AS total_amount FROM Orders GROUP BY customer_id;
```

67. Find customers who have placed more than 3 orders:

```
SELECT customer_id, COUNT(*) AS order_count FROM Orders GROUP BY customer_id  
HAVING COUNT(*) > 3;
```

68. Find the average order amount for each customer:

```
SELECT customer_id, AVG(amount) AS avg_amount FROM Orders GROUP BY customer_id;
```

69. Find the highest order amount placed by each customer:

```
SELECT customer_id, MAX(amount) AS max_amount FROM Orders GROUP BY customer_id;
```

70. Display customers sorted by their total purchase amount:

```
SELECT c.customer_id, c.customer_name, SUM(o.amount) AS total_purchase FROM  
Customers c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id,  
c.customer_name ORDER BY total_purchase DESC;
```

71. Find customers whose total purchase amount exceeds 10,000:

```
SELECT c.customer_id, c.customer_name, SUM(o.amount) AS total_purchase FROM  
Customers c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id,  
c.customer_name HAVING SUM(o.amount) > 10000;
```

72. Display customer names along with the total number of orders placed:

```
SELECT c.customer_name, COUNT(o.order_id) AS total_orders FROM Customers c LEFT JOIN  
Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id, c.customer_name;
```

73. Find the customer who spent the highest amount:

```
SELECT c.customer_id, c.customer_name, SUM(o.amount) AS total_spent FROM Customers  
c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id,  
c.customer_name ORDER BY total_spent DESC LIMIT 1;
```

74. Find the customer who placed the maximum number of orders:

```
SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS order_count FROM  
Customers c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id,  
c.customer_name ORDER BY order_count DESC LIMIT 1;
```

75. Find customers whose average order amount is greater than 2,000:

```
SELECT customer_id, AVG(amount) AS avg_amount FROM Orders GROUP BY customer_id  
HAVING AVG(amount) > 2000;
```

76. Display the top 5 customers based on total purchase amount:

```
SELECT c.customer_id, c.customer_name, SUM(o.amount) AS total_purchase FROM  
Customers c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id,  
c.customer_name ORDER BY total_purchase DESC LIMIT 5;
```

77. Find the minimum order amount for each customer:

```
SELECT customer_id, MIN(amount) AS min_amount FROM Orders GROUP BY customer_id;
```

78. Find customers who have placed orders worth more than 5,000 in total:

```
SELECT customer_id, SUM(amount) AS total_spent FROM Orders GROUP BY customer_id  
HAVING SUM(amount) > 5000;
```

79. Display customer-wise total orders and total purchase amount:

```
SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders,  
SUM(o.amount) AS total_purchase FROM Customers c LEFT JOIN Orders o ON c.customer_id  
= o.customer_id GROUP BY c.customer_id, c.customer_name;
```

80. Find customers who placed more than 2 orders and spent more than 8,000:

```
SELECT customer_id, COUNT(*) AS order_count, SUM(amount) AS total_spent FROM Orders  
GROUP BY customer_id HAVING COUNT(*) > 2 AND SUM(amount) > 8000;
```

## **SECTION E: STUDENTS TABLE**

81. Find the average marks scored by students in each department:

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department;
```

82. Find departments whose average marks are above 75:

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department  
HAVING AVG(marks) > 75;
```

83. Find the highest mark scored in each department:

```
SELECT department, MAX(marks) AS max_marks FROM Students GROUP BY department;
```

84. Find the total number of students in each department:

```
SELECT department, COUNT(*) AS total_students FROM Students GROUP BY department;
```

85. Find departments having more than 5 students:

```
SELECT department, COUNT(*) AS student_count FROM Students GROUP BY department  
HAVING COUNT(*) > 5;
```

86. Display departments sorted by average marks in descending order:

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department  
ORDER BY avg_marks DESC;
```

87. Find the top 3 departments based on average marks:

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department  
ORDER BY avg_marks DESC LIMIT 3;
```

88. Find departments whose average marks are between 70 and 90:

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department  
HAVING AVG(marks) BETWEEN 70 AND 90;
```

89. Find the total marks scored by students in each department:

```
SELECT department, SUM(marks) AS total_marks FROM Students GROUP BY department;
```

90. Display departments sorted by the total number of students:

```
SELECT department, COUNT(*) AS total_students FROM Students GROUP BY department  
ORDER BY total_students DESC;
```

91. Find the lowest mark scored in each department:

```
SELECT department, MIN(marks) AS min_marks FROM Students GROUP BY department;
```

92. Find departments where the highest mark is greater than 90:

```
SELECT department, MAX(marks) AS max_marks FROM Students GROUP BY department  
HAVING MAX(marks) > 90;
```

93. Find the number of students scoring above 80 in each department:

```
SELECT department, COUNT(*) AS student_count FROM Students WHERE marks > 80 GROUP  
BY department;
```

94. Find departments where more than 3 students scored above 75:

```
SELECT department, COUNT(*) AS student_count FROM Students WHERE marks > 75 GROUP  
BY department HAVING COUNT(*) > 3;
```

95. Display departments ordered by highest mark in descending order:

```
SELECT department, MAX(marks) AS max_marks FROM Students GROUP BY department  
ORDER BY max_marks DESC;
```